

APUNTES DE BASES DE DATOS (RAs 2, 3 y 4)

RA2: DISEÑO FÍSICO RELACIONAL

1. Vistas

- **Definición:** Una vista es una "tabla virtual". No almacena datos físicamente; ejecuta una consulta SELECT cada vez que se usa.
- **Sintaxis:** `CREATE OR REPLACE VIEW Nombre_Vista AS SELECT ...;`
- **Restricciones de control:**
 - `WITH CHECK OPTION`: Solo permite INSERT/UPDATE si el resultado sigue apareciendo en la vista (cumple el WHERE).
 - `WITH READ ONLY`: Prohíbe cualquier cambio (DML), solo permite consultas.
- **Modificación de datos:** Solo en vistas simples (una sola tabla, sin funciones). Para insertar, la vista debe incluir todas las columnas NOT NULL de la tabla base que carezcan de valor por defecto.
- `DROP VIEW nombre;` para borrarlas

RA3: REALIZACIÓN DE CONSULTAS

2. Consultas simples y Pseudocolumnas

- **Estructura:** `SELECT [DISTINCT] columnas FROM tabla WHERE condiciones ORDER BY col [ASC|DESC];`
- **Filtros de texto (LIKE):** % (varios caracteres) y _ (un carácter exacto).
- **Pseudocolumnas (Identificadores invisibles):**
 - **ROWID:** Dirección física única del registro en el disco. No cambia nunca.
 - **ROWNUM:** Número de orden temporal asignado al extraer la fila. Se usa para limitar: `WHERE ROWNUM <= N`.
- **Tratamiento de Nulos:** Recuerda que NULL no es igual a nada (ni a cero ni a vacío). Se usa `IS NULL` o `IS NOT NULL`.

3. JOINS (Inner y Outer)

- **Estándar SQL 1999:**
 - INNER JOIN: Solo filas con pareja en ambas tablas.
 - LEFT/RIGHT/FULL OUTER JOIN: Incluye filas "huérfanas" de la tabla izquierda, derecha o ambas.
 - **CROSS JOIN:** Es el **Producto Cartesiano**. Une cada fila de A con todas las de B.
 - *Nota de examen:* Si en un SELECT con varias tablas olvidas la condición de unión (el ON o el WHERE), el resultado es un producto cartesiano.
- **Estándar SQL 1992:** Se pone el signo (+) en el lado de la tabla que NO tiene los datos para forzar la aparición de nulos.

4. Consultas resumen (Funciones de agregación)

- **Funciones:** COUNT(), SUM(), AVG(), MIN(), MAX().
- **EL MATIZ DE LOS NULOS:** Las funciones AVG, SUM, MIN y MAX **ignoran los valores NULL** automáticamente. Solo COUNT(*) cuenta todas las filas incluyendo las que tienen nulos.
- **Agrupación:** GROUP BY crea los grupos; HAVING filtra esos grupos ya creados.
 - *Regla de oro:* Cualquier columna que pongas en el SELECT y no tenga una función (como SUM o AVG) **debe** estar en el GROUP BY.

5. Subconsultas

- **Fila simple** (Devuelve 1 valor): SELECT * FROM Emp WHERE Salario > (SELECT AVG(Salario) FROM Emp);
- **Múltiples filas** (Devuelve una lista): SELECT Nombre FROM Emp WHERE Dept_ID IN (SELECT Dept_ID FROM Dept WHERE Loc = 'BCN');
- **Correlacionada** (Usa datos de la tabla de fuera): SELECT Nombre FROM Emp e WHERE Salario > (SELECT AVG(Salario) FROM Emp WHERE Dept_ID = e.Dept_ID);

6. Múltiples selecciones (Operadores de conjuntos)

- **Operadores:** UNION (sin duplicados), UNION ALL (con duplicados), INTERSECT (comunes), MINUS (diferencia).
- **REQUISITO CRÍTICO:** Para que funcionen, ambos SELECT deben tener **exactamente el mismo número de columnas** y los **tipos de datos deben ser compatibles** en el mismo orden de posición.

RA4: TRATAMIENTO DE DATOS Y TRANSACCIONES

7. Insert, update y delete (DML)

- `INSERT INTO tabla (cols) VALUES (vals);`
- `UPDATE tabla SET col = val WHERE condicion;`
- `DELETE FROM tabla WHERE condicion;`
- **ADVERTENCIA DE EXAMEN:** Si ejecutas un UPDATE o DELETE **sin la cláusula WHERE**, la acción se aplicará a **TODOS** los registros de la tabla. Es un error crítico muy penalizado.

8. Insert con consulta

- **Sintaxis:** `INSERT INTO Tabla_Destino SELECT * FROM Tabla_Origen WHERE ...;`
- **Clave:** En este comando **NUNCA se usa la palabra VALUES**.

9. Guiones

- **Definición:** Un archivo con una serie de comandos SQL ordenados para realizar un proceso completo de principio a fin (ej. crear tablas, insertar datos base y confirmar cambios).

10. Rollback y savepoint

- `ROLLBACK ;`: Deshace todo hasta el último COMMIT y termina la transacción.
- `SAVEPOINT punto1 ;`: Crea una "bandera" o marca en el tiempo.
- **REGLA DE ORO:** `ROLLBACK TO SAVEPOINT punto1 ;` deshace solo lo ocurrido después de esa marca, pero **la transacción sigue abierta**. Debes terminarla después con un COMMIT o un ROLLBACK total.

11. Commit

- **Commit Explícito:** El comando `COMMIT ;` guarda los cambios de forma permanente.
- **PELIGRO: Commit Implícito:** Cualquier comando de estructura (**DDL** como `CREATE`, `ALTER`, `DROP` o `TRUNCATE`) ejecuta un COMMIT automático. Si haces un `UPDATE` y justo después un `CREATE TABLE`, el `UPDATE` se guarda permanentemente y ya no podrás deshacerlo.

12. Medidas de integridad al suprimir

- **Restrict (No permitir):** Valor por defecto. Si intentas borrar un padre con hijos, da error.
- **On Delete Cascade (Cascada):** Al borrar el registro padre, Oracle borra automáticamente todos sus registros hijos relacionados.
- **On Delete Set Null (Poner a nulo):** Al borrar el padre, los hijos se mantienen pero su campo de unión (FK) se vuelve NULL.

13. Secuencias

- **Definición:** Objetos que generan números enteros únicos de forma secuencial. Ideales para Claves Primarias (IDs).
- **Creación:** `CREATE SEQUENCE Nombre_Seq START WITH 1 INCREMENT BY 1;`
- **Uso de Pseudometodos:**
 - **NEXTVAL:** Devuelve el siguiente número disponible. Se usa en el `INSERT`.
 - **CURRVAL:** Devuelve el último número generado en la sesión actual.
- **Ejemplo en INSERT:** `INSERT INTO Empleados (ID, Nombre) VALUES (Nombre_Seq.NEXTVAL, 'Juan');`
- **Borrado:** `DROP SEQUENCE Nombre_Seq;`
- **Nota:** Las secuencias no están ligadas a una tabla concreta; una misma secuencia puede alimentar varias tablas si se desea.